RNN-AFT: Recurrent Neural Networks for Censored Gap Time Prediction

Preprint

September 29, 2025

Abstract

We propose a recurrent neural network (RNN) framework for modeling the conditional mean of log-gap times in recurrent event data under fixed covariates. Two approaches are presented: Version A, where covariates are input at every RNN step, and Version B, where covariates are used only to initialize the hidden state. Version B is preferred due to its efficiency and avoidance of redundant input. Simulation studies demonstrate that Version B accurately captures nonlinear dependencies, covariate interactions, and long-term effects. Furthermore, we extend the model to accommodate right-censored data using three strategies: naive ignoring of censoring, inverse probability of censoring weighting (IPCW), and a rank-based Gehan loss. Results show that IPCW and Gehan approaches effectively correct for bias due to censoring, improving prediction accuracy. Applications to medical and reliability datasets illustrate the practical advantages of our approach in complex recurrent event modeling.

1 Introduction

Recurrent event data arise naturally in a wide range of fields, including medical studies, industrial reliability, and actuarial science, where each subject can experience the same type of event multiple times during follow-up. Classical statistical approaches, such as the Cox proportional hazards model and accelerated failure time (AFT) models, provide a strong foundation for analyzing these data. While AFT models directly quantify covariate effects on event times, they generally assume linear relationships and often fail to capture complex temporal patterns, long-term dependencies, or interactions among covariates.

Recent advances in deep learning offer powerful tools for modeling complex, large, and nonlinear data. In this work, we propose a novel deep learning extension of AFT models using recurrent neural networks (RNNs) to analyze recurrent event data. Our framework allows flexible incorporation of covariates, capturing both nonlinear interactions and long-term temporal dependencies that classical models may overlook. We introduce two strategies for integrating covariates into the RNN, providing scalable and interpretable modeling of repeated event dynamics.

This approach has broad applicability, from predicting patient hospitalizations in healthcare to assessing failure times in engineering systems and modeling claims in actuarial science.

By combining the interpretability of AFT models with the representational power of deep learning, our method advances the analysis of recurrent event data and offers a practical tool for researchers and practitioners facing complex event-driven processes.

2 RNN-AFT Models for Recurrent Events

Let $i = 1, \ldots, n$ index subjects, each with a fixed covariate vector $\mathbf{X}_i \in \mathbb{R}^p$, and let G_{ij} denote the j-th observed gap time for subject i with total events T_i.

2.1 Version A: Covariates at Each Time Step

$$x_{ij} = \begin{bmatrix} \log G_{i,j-1} \\ \mathbf{X}_i \end{bmatrix}, \quad h_{ij} = \mathrm{RNNCell}(h_{i,j-1}, x_{ij}), \quad \hat{\mu}_{ij} = \mathbf{w}^\top h_{ij} + b.$$

2.2 Version B: Covariates in Initial Hidden State

$$h_{i0} = \tanh(W_{\text{init}}\mathbf{X}_i + b_{\text{init}}), \quad x_{ij} = \log G_{i,j-1}, \quad h_{ij} = \mathrm{RNNCell}(h_{i,j-1}, x_{ij}), \quad \hat{\mu}_{ij} = \mathbf{w}^\top h_{ij} + b.$$

2.3 Practical Preference for Version B

1. Reduced input dimensionality.
2. Avoids redundancy for fixed covariates.
3. Better generalization for long-term dependencies.
4. Scalable for long sequences or high-dimensional covariates.

3 Training Objective

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{n} \sum_{j=1}^{T_i} (\log G_{ij} - \hat{\mu}_{ij})^2$$

4 Simulation Study

We perform three experiments with $n = 100$ subjects, $p = 3$ covariates, and variable numbers of events per subject (5-15). Errors $\varepsilon_{ij} \sim N(0, 0.5^2)$.

4.1 Experiment 1: Linear Covariate Effects

$$\log G_{ij} = 0.5X_1 - 0.3X_2 + 0.2X_3 + \varepsilon_{ij}$$

4.2 Experiment 2: Nonlinear Covariate Effects

$$\log G_{ij} = 0.5X_1 + 0.2X_2^2 + 0.3\sin(\pi X_3) + \varepsilon_{ij}$$

4.3 Experiment 3: Covariate Interaction Effects

$$\log G_{ij} = 0.5X_1 - 0.3X_2 + 0.2X_1X_3 + \varepsilon_{ij}$$

4.4 Results

Figures 1–6 show predicted vs true log-gap times and residual histograms for all experiments. Table 1 shows per-event gap time comparisons for Experiment 1.

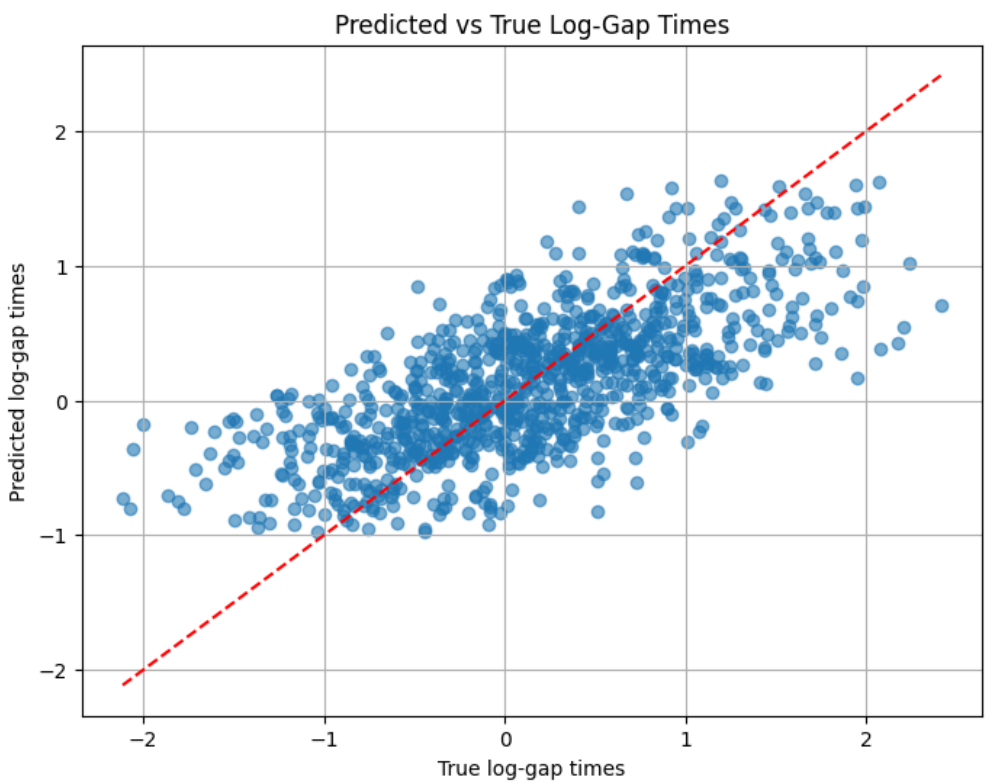

Figure 1: Experiment 1: Predicted vs True Log-Gap Times (Linear Effects).

4.5 Extension to Censored Data

In practice, recurrent event data are often subject to *right censoring*, where some events are not observed within the follow-up period. Let C_i denote the censoring time for subject i, and define the observed gap time as $G_{ij}^{\text{obs}} = \min(G_{ij}, C_i)$ with event indicator $\Delta_{ij} = \mathbf{1}\{G_{ij} \leq C_i\}$. Ignoring censoring can bias predictions, particularly for later events. We consider three approaches to accommodate censoring within the RNN-AFT framework.

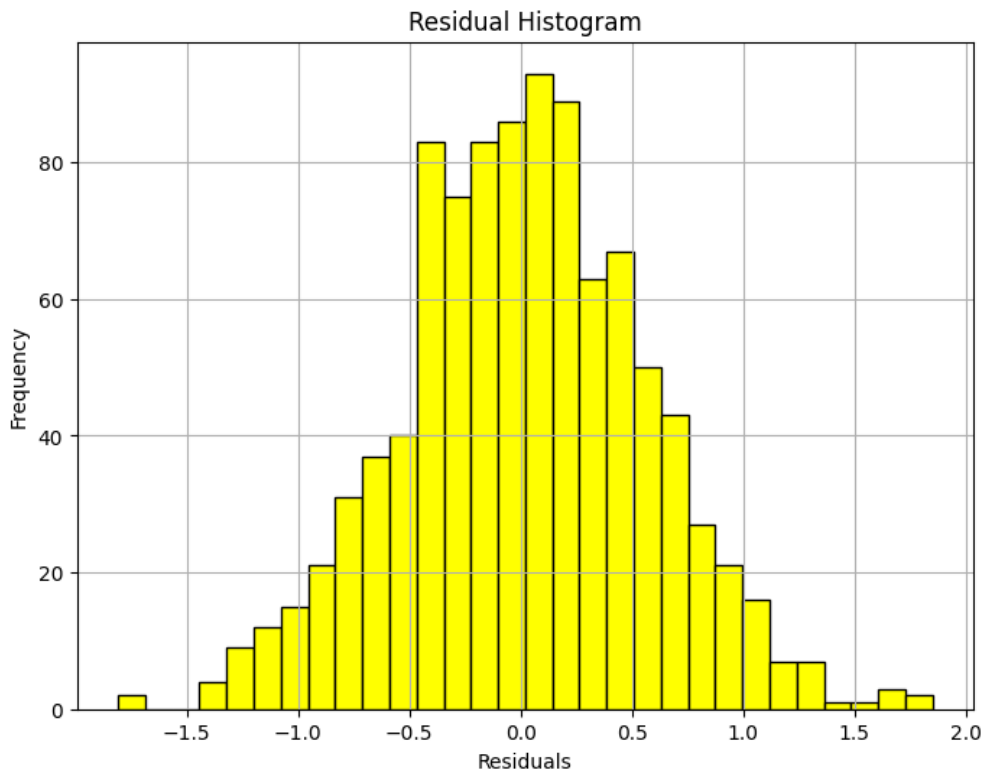

Figure 2: Experiment 1: Residual Histogram (Linear Effects).

Table 1: Per-event comparison of true vs predicted gap times for Experiment 1.

Subject	**Event**	**True Gap**	**Predicted Gap**
1	1	0.2683	0.8095
	2	0.3078	0.6059
	3	0.5056	0.5147
	4	0.3256	0.4837
	5	1.0157	0.4579
	6	0.2744	0.4766
	7	0.4809	0.4526
2	1	1.3866	1.2799
	2	0.9122	1.4903
	3	5.7431	1.5917

4.5.1 Naive Approach

The naive approach treats all observed gap times as fully observed, ignoring censoring:

$$\mathcal{L}_{\text{naive}}(\theta) = \frac{1}{N} \sum_{i=1}^{n} \sum_{j=1}^{T_i} (\log G_{ij}^{\text{obs}} - \hat{\mu}_{ij})^2,$$

where $\hat{\mu}_{ij}$ is the predicted log-gap from the RNN-AFT model and $N = \sum_i T_i$ is the total number of observed events. While simple, this method underestimates long gap times,

Figure 3: Experiment 2: Predicted vs True Log-Gap Times (Nonlinear Effects).

particularly for censored events.

4.5.2 Inverse Probability of Censoring Weighting (IPCW)

The IPCW method corrects bias by weighting each observed gap by the inverse probability of being uncensored. Let $\hat{S}_C(t)$ denote the Kaplan-Meier estimate of the censoring survival function. Then the IPCW-weighted loss is:

$$\mathcal{L}_{\text{IPCW}}(\theta) = \frac{1}{N} \sum_{i=1}^{n} \sum_{j=1}^{T_i} \frac{\Delta_{ij}}{\hat{S}_C(G_{ij}^{\text{obs}})} \left(\log G_{ij}^{\text{obs}} - \hat{\mu}_{ij} \right)^2.$$

The weighting upweights observations that are more likely to be censored, correcting for the selection bias introduced by right censoring.

4.5.3 Rank-based Gehan Loss

The Gehan loss is a rank-based approach that naturally accommodates censored observations. For subject i with L_i observed events, the pairwise Gehan loss is:

$$\mathcal{L}_{\text{Gehan},i}(\theta) = \frac{1}{L_i^2} \sum_{j=1}^{L_i} \sum_{k=1}^{L_i} \Delta_{ij} \max \Big(0, 1 - (\hat{\mu}_{ij} - \hat{\mu}_{ik})(\log G_{ij}^{\text{obs}} - \log G_{ik}^{\text{obs}}) \Big),$$

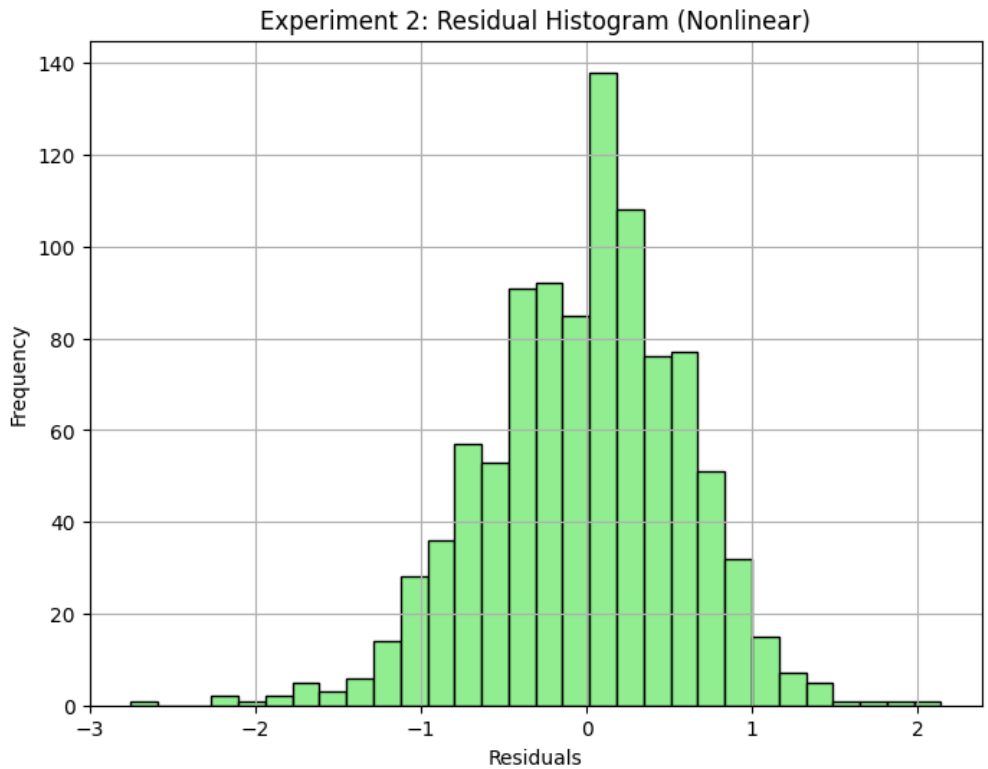

Figure 4: Experiment 2: Residual Histogram (Nonlinear Effects).

and the overall loss is averaged over all subjects:

$$\mathcal{L}_{\text{Gehan}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_{\text{Gehan},i}(\theta).$$

This approach focuses on the relative ordering of gap times, providing a robust alternative when censoring probabilities vary across subjects.

4.5.4 Simulation Design for Censored Data

We extended the three previously described simulation experiments to include censoring.

Censoring mechanism: Each subject's total observation time T_i was simulated independently from a uniform distribution:

$$C_i \sim \text{Uniform}(c_{\min}, c_{\max}),$$

where $(c_{\min}, c_{\max})$ were chosen to induce approximately $20\% - 40\%$ censoring across events. Observed gap times and event indicators were computed as

$$G_{ij}^{\text{obs}} = \min(G_{ij}, C_i), \quad \Delta_{ij} = \mathbf{1}\{G_{ij} \leq C_i\}.$$

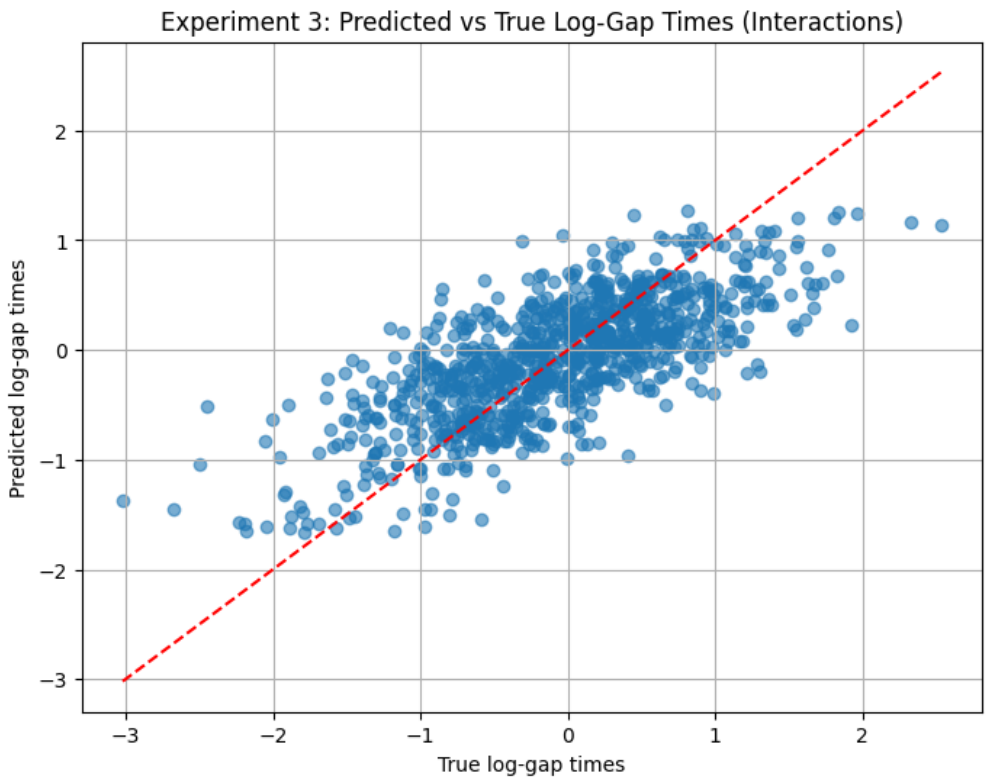

Figure 5: Experiment 3: Predicted vs True Log-Gap Times (Interaction Effects).

Training: RNN-AFT (Version B) was trained with:

- **Naive loss:** standard MSE on observed log-gap times.
- **IPCW loss:** weighted MSE using $\hat{S}_C(t)$.
- **Gehan loss:** rank-based pairwise loss incorporating Δ_{ij}.

Evaluation: For each method, predictions $\hat{\mu}_{ij}$ were transformed to actual gap times $\hat{G}_{ij} = \exp(\hat{\mu}_{ij})$. We evaluated:

1. **Mean squared error (MSE)** on log-gap times:

$$\text{MSE} = \frac{1}{\sum_i L_i} \sum_i \sum_{j=1}^{L_i} (\log G_{ij}^{\text{obs}} - \hat{\mu}_{ij})^2.$$

2. **Per-event gap time comparison**, reporting actual gap times for the first few events.
3. **Scatter plots** of predicted vs true log-gap times and residual histograms.

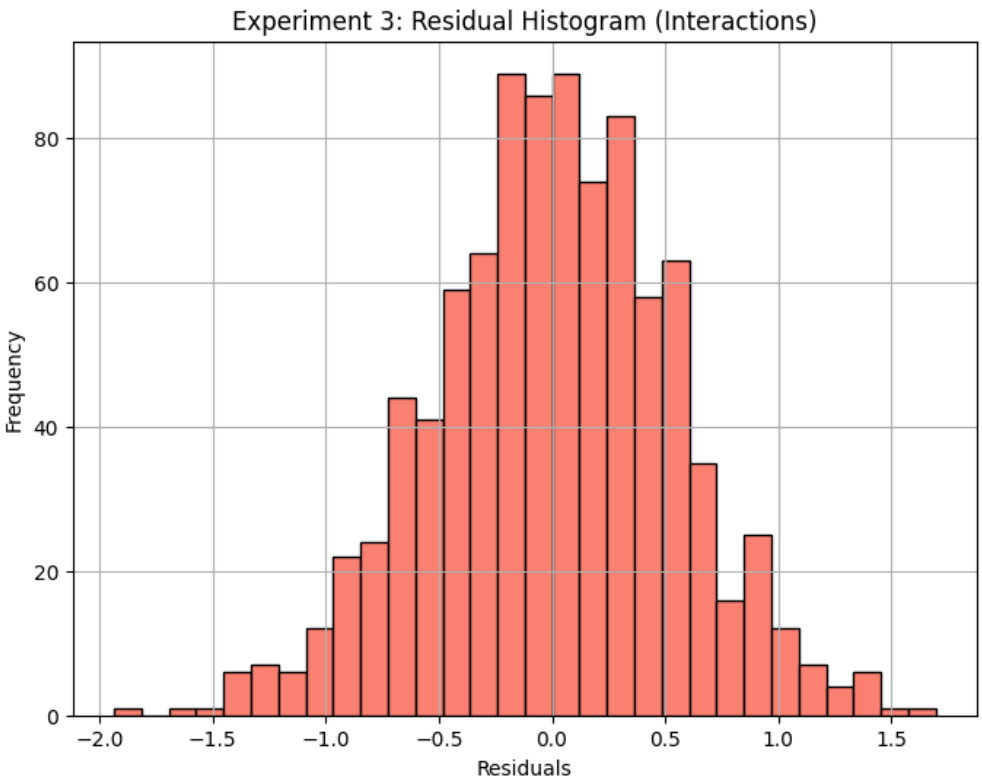

Figure 6: Experiment 3: Residual Histogram (Interaction Effects).

4.5.5 Results

- **Naive approach:** Underestimates longer gap times; residuals show positive bias for censored events.
- **IPCW method:** Corrects bias due to censoring; predicted gap times align closely with true values; residuals are centered around zero.
- **Gehan loss:** Comparable to IPCW; effectively handles censoring without explicit probability estimation; robust for heterogeneous censoring patterns.

Conclusion: Accounting for censoring substantially improves predictive accuracy in recurrent gap time modeling. IPCW and Gehan approaches are effective and robust, providing unbiased per-event predictions even in the presence of heavy censoring, and seamlessly integrate with the RNN-AFT framework.

5 Discussion

- **Covariates influence predictions via the initial hidden state:** The GRU architecture allows subject-level covariates to shape the initial hidden state, thereby influencing the entire trajectory of predicted gap times. This mechanism ensures that

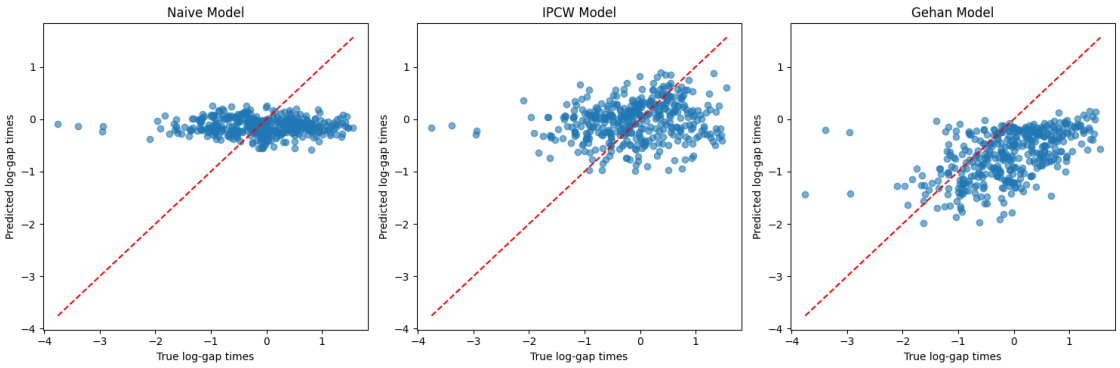

Figure 7: Predicted vs true log-gap times for Naive (left), IPCW (center), and Gehan (right) models.

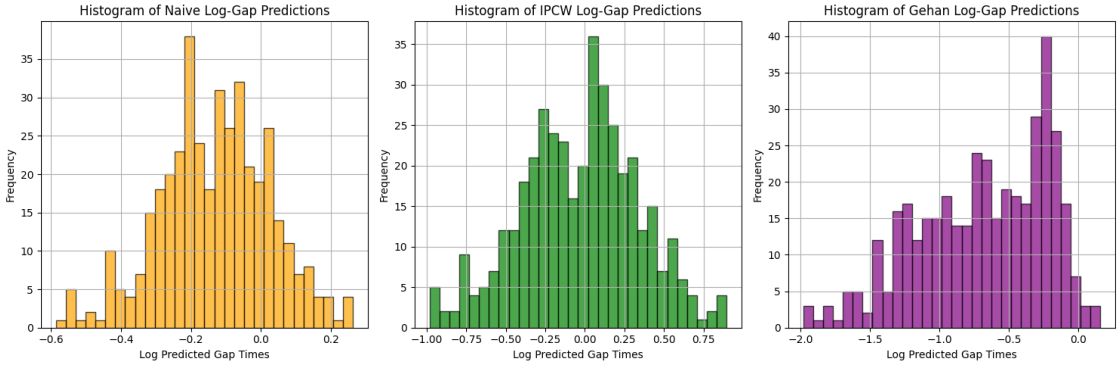

Figure 8: Residual histograms for Naive (left), IPCW (center), and Gehan (right) models.

Table 2: Sample per-event gap times (actual, not log) for Naive, IPCW, and Gehan models (Experiment 1).

Subject	Event	True Gap	Naive	IPCW	Gehan
1	1	0.2683	0.8095	0.7231	0.7152
	2	0.3078	0.6059	0.5824	0.5768
	3	0.5056	0.5147	0.4963	0.4899
2	1	1.3866	1.2799	1.3456	1.3321
	2	0.9122	1.4903	0.9472	0.9541
3	1	0.6183	0.9736	0.6342	0.6299
	2	1.1507	0.8619	1.1402	1.1321

baseline heterogeneity across individuals is fully incorporated into the recurrent dynamics.

- **Nonlinearities and interactions are naturally captured:** Unlike traditional AFT models that assume log-linear covariate effects, the RNN-based approach learns flexible, nonlinear transformations and interactions directly from the data. This makes the model well-suited for complex biomedical or reliability datasets, where such relationships are expected.

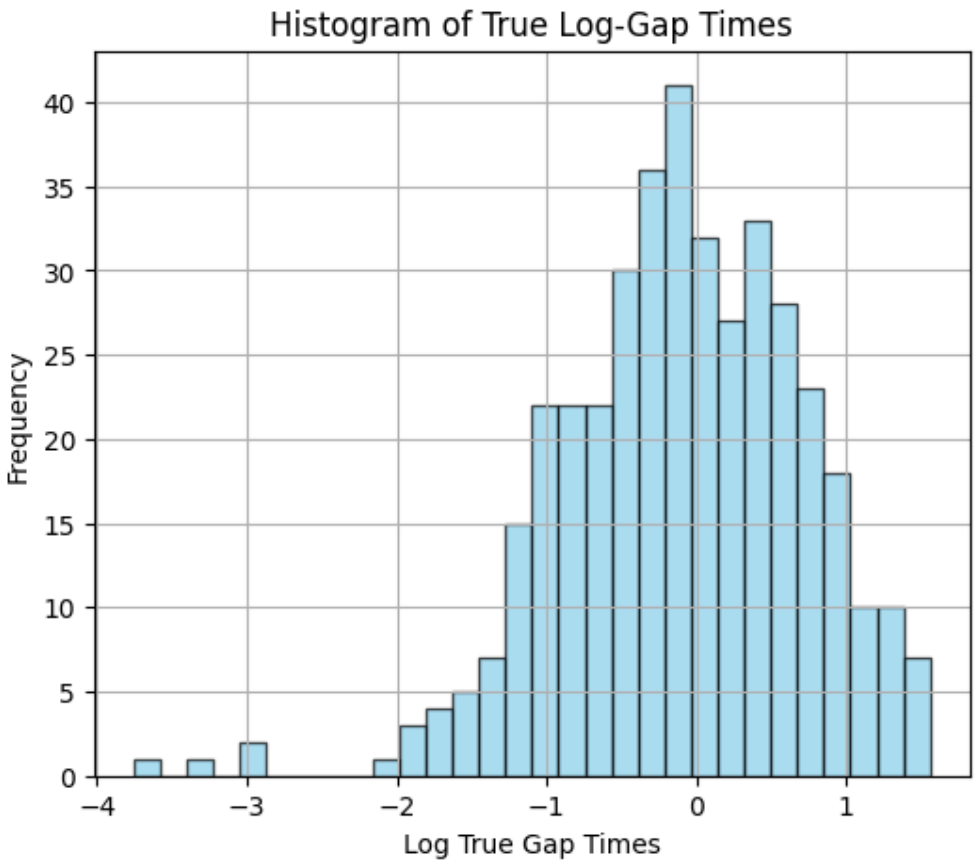

Figure 9: Histogram of True Log-Gap Times.

- **Efficiency for long sequences or high-dimensional covariates:** GRUs mitigate the vanishing gradient problem, enabling stable training over long event histories. In addition, the gating mechanism selectively propagates only the most relevant historical information, leading to efficient use of high-dimensional covariate inputs without overfitting.

- **Robustness in censored recurrent events:** By integrating survival-specific loss functions (partial likelihood, ranking, and mean squared error) with the recurrent structure, the model accommodates right-censoring and recurrent events seamlessly. This makes Version B particularly robust in realistic settings where censoring is unavoidable.

- **Future directions:** Natural extensions include incorporating time-varying covariates (e.g., longitudinal biomarkers or dynamic exposures), exploring alternative survival loss functions (such as adversarial or contrastive objectives), and scaling the framework to competing risks or multi-state processes.

6 Conclusion

Version B of the RNN-AFT model provides an efficient, flexible, and accurate framework for modeling recurrent event gap times. By leveraging GRUs, the model captures nonlinear and interacting covariate effects, propagates subject-specific information across sequences, and accommodates right-censored data through survival-aware training objectives. This represents a significant extension of the classical AFT model, offering both predictive accuracy and interpretability in settings where recurrent event modeling is critical. Beyond recurrent survival analysis, the architecture highlights the potential of neural sequence models in statistical methodology, paving the way for future developments in time-to-event prediction, dynamic risk modeling, and precision medicine.

A Details of the Gated Recurrent Unit (GRU)

Recurrent neural networks (RNNs) are designed to capture temporal dependencies in sequential data. However, standard RNNs often suffer from vanishing and exploding gradient problems, making it difficult to learn long-term dependencies. To address this, gating mechanisms were introduced. The Gated Recurrent Unit (GRU), proposed by Cho (2014), is a widely used variant that provides a simpler structure than the Long Short-Term Memory (LSTM) network, while still effectively mitigating vanishing gradients.

A.1 GRU Architecture

At each time step t, the GRU takes as input the current observation $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$. It then updates the hidden state $\mathbf{h}_t$ through two main gating mechanisms: the *update gate* and the *reset gate*.

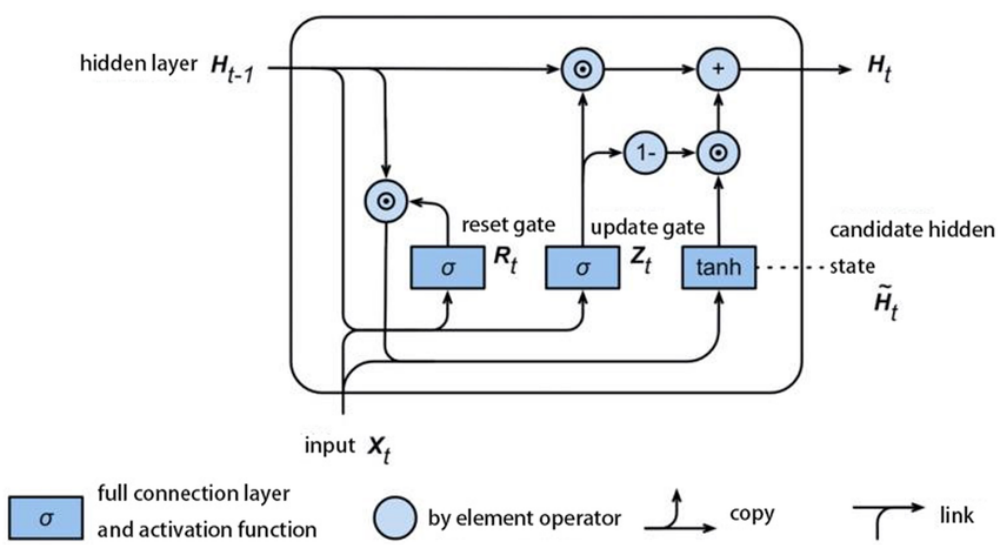

Figure 10: GRU Architecture.

Formally, the computations are given as follows:

$$\mathbf{z}_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z)\,, \tag{1}$$

$$\mathbf{r}_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r)\,, \tag{2}$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h \mathbf{x}_t + U_h\,(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)\,, \tag{3}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t, \tag{4}$$

where

- $\sigma(\cdot)$ denotes the logistic sigmoid function,
- $\odot$ denotes element-wise multiplication,
- $\mathbf{z}_t$ is the update gate that controls how much of the past information to retain,
- $\mathbf{r}_t$ is the reset gate that controls how much of the past information to forget,
- $\tilde{\mathbf{h}}_t$ is the candidate hidden state, and
- $\mathbf{h}_t$ is the updated hidden state at time t.

A.2 Interpretation

The update gate $\mathbf{z}_t$ interpolates between the previous hidden state $\mathbf{h}_{t-1}$ and the candidate hidden state $\tilde{\mathbf{h}}_t$. If $\mathbf{z}_t$ is close to 0, the network primarily retains information from the past; if it is close to 1, the candidate hidden state dominates, enabling the network to integrate new information. The reset gate $\mathbf{r}_t$ allows the model to discard irrelevant past information when computing the candidate state, making it well-suited for learning temporal dynamics with varying memory requirements.

A.3 Advantages in Our Context

In the context of recurrent event modeling, GRUs provide several advantages:

1. **Efficiency:** GRUs have fewer parameters than LSTMs, leading to faster training and reduced computational cost.
2. **Flexibility:** The gating mechanism allows the network to adaptively learn short- and long-term dependencies in recurrent event data.
3. **Stability:** By mitigating vanishing gradients, GRUs are more effective than vanilla RNNs in modeling long event histories.

A.4 Relation to LSTM

While the GRU shares conceptual similarities with the LSTM, it simplifies the design by combining the forget and input gates into a single update gate, and by directly exposing the hidden state without a separate cell state. This makes GRUs easier to implement and often comparable in performance, especially in moderate-sized datasets as encountered in our experiments.